



《数据库原理及应用》

第6章 完整性约束（自学）

计算机科学与工程学院（网络空间安全学院）

任课教师：胡旺

2025年12月16日星期二



本章内容



1

数据库完整性概述

2

完整性约束的分类

3

完整性约束的定义

4

完整性约束的修改

5

*完整性约束的验证

6

*数据库产品的完整性支持



完整性概述



- 完整性约束是加在数据库模式上的一个具体条件，它规定什么样的数据能够存储到数据库系统中。
- 例如：
 - ◆ 学生的年龄必须是整数，取值范围为14--29；
 - ◆ 学生的性别只能是“男”或“女”；
 - ◆ 学生的学号一定是唯一的；
 - ◆ 学生所在的系必须是学校开设的系；



完整性概述



- 数据完整性和安全性是两个不同概念
- 数据的**完整性**是为了防止数据库中存在不符合语义的数据，防止错误数据的输入和输出所造成的无效操作和错误结果。
- 数据的**安全性**是防止非法用户的非法操作所造成的对数据库的恶意破坏。



■ 完整性约束条件定义机制

- ◆ 完整性约束条件：数据模型的组成部分，约束数据库中数据的语义
- ◆ DBMS应提供定义数据库完整性约束条件，并把它们作为模式的一部分存入数据库中

■ 完整性检查机制

- ◆ 检查用户发出的操作请求是否违背了完整性约束条件

■ 违约反应

- ◆ 如果发现用户的操作请求使数据违背了完整性约束条件，则采取一定的动作来保证数据的完整性



完整性约束分类



■ 按照完整性约束条件作用的对象分：

- ◆ 类型约束
- ◆ 属性约束
- ◆ 关系变量约束
- ◆ 数据库约束

■ 按照完整性约束条件声明时的位置分：

- ◆ 列级约束
- ◆ 表级约束

■ 两种状态

- ◆ 静态：对静态对象的约束是反映数据库状态合理性的约束
- ◆ 动态：对动态对象的约束是反映数据库状态变迁的约束，新旧值之间满足的约束



■ Primary Key约束：实体完整性规则？

■ 列级约束

```
CREATE TABLE RecipeMaster {  
  Rno VARCHAR(10) PRIMARY KEY,  
  DGno VARCHAR(10),  
  Rdatetime DATETIME  
}
```

■ 表级约束：单属性主键

```
CREATE TABLE Medicine {  
  Mno VARCHAR(10),  
  Mname VARCHAR(50) NOT NULL,  
  Mprice DECIMAL(18, 2) NOT NULL,  
  Munit VARCHAR(10),  
  Mtype VARCHAR(10),  
  PRIMARY KEY (Mno)  
}
```



■ 表级约束：多属性组合主键

```
CREATE TABLE RecipeDetail {  
  Rno VARCHAR(10),  
  Mno VARCHAR(10) NOT NULL,  
  Mamount DECIMAL(18, 0),  
  PRIMARY KEY (Rno, Mno)  
}
```

■ UNIQUE约束定义和PRIMARY KEY约束定义不能在同一属性上

■ PRIMARY KEY子句中的每个属性的取值都必须是NOT NULL



■ 列级约束

```
CREATE TABLE Dept {  
  DeptNo VARCHAR(10) PRIMARY KEY,  
  DeptName VARCHAR(50) UNIQUE,  
  ParentDeptNo VARCHAR(10),  
  Manager VARCHAR(10)  
}
```

■ 索引对象约束

```
CREATE UNIQUE INDEX deptname_index ON dept (DeptName)
```

■ UNIQUE与Primary Key

- ◆ 在一个关系中，PRIMARY KEY只有一个，而UNIQUE可以声明多个
- ◆ PRIMARY KEY要求属性取值不能为NULL，而UNIQUE允许属性取空值，允许多个空值同时存在
- ◆ 在定义了UNIQUE，PRIMARY KEY 约束的属性上建立索引是十分必要的，它可以使约束的检查执行起来更有效



■ NOT NULL列级约束

```
CREATE TABLE Diagnosis{  
  DGno VARCHAR(10) PRIMARY KEY,  
  Pno VARCHAR(10) NOT NULL,  
  Dno VARCHAR(10) NOT NULL,  
  Symptom VARCHAR(100),  
  Diagnosis VARCHAR(100),  
  DGtime DATETIME,  
  Rfee DECIMAL(18,2) NOT NULL  
}
```

■ NOT NULL域对象约束

```
CREATE DOMAIN fee DECIMAL(18,2) NOT NULL;
```



■ CHECK表级约束

```
CREATE TABLE Doctor {  
  Dno VARCHAR(10),  
  Dname VARCHAR(50) NOT NULL,  
  Dsex VARCHAR(2),  
  Dage INT,  
  Ddeptno VARCHAR(10),  
  Dlevel VARCHAR(50),  
  Dsalary DECIMAL(18, 2),  
  PRIMARY KEY(Dno),  
  CHECK( Dsex IN ( '男' , '女' )),  
  CHECK( Dage > 0 AND Dage <60)  
}
```



■ CHECK域对象约束

```
CREATE DOMAIN rfee DECIMAL(18, 2)
CONSTRAINT rfee_test CHECK(VALUE >0)
```

■ CHECK 在SQL条件中的约束

```
CREATE TABLE RecipeDetail{
Rno VARCHAR(10),
Mno VARCHAR(10) NOT NULL,
Mamout DECIMAL(18, 0),
PRIMARY KEY(Rno, Mno),
CHECK (Mno IN (SELECT Mno FROM medicine))
}
```

- ◆ 上述CHECK语句实际上是定义了一个参照完整性约束
- ◆ 一般而言，由于Check条件中允许出现包含其他关系的子查询，Check条件可能变得更复杂，也更难检测。



■ FK的值必须为（参照完整性？）：

- ◆ 取空值；
- ◆ 等于对应PK中某个主码值。

■ FK列级约束

```
CREATE TABLE Doctor {  
  Dno VARCHAR(10),  
  Dname VARCHAR(50) NOT NULL,  
  Dsex VARCHAR(2),  
  Dage INT,  
  Ddeptno VARCHAR(10) REFERENCES Dept (DeptNo),  
  Dlevel VARCHAR(50),  
  Dsalary DECIMAL(18, 2),  
  PRIMARY KEY (Dno),  
  CHECK ( Dsex IN ('男', '女'))  
}
```



■ FK表级约束

```
CREATE TABLE RecipeDetail {  
  Rno VARCHAR(10),  
  Mno VARCHAR(10) NOT NULL,  
  Mamount DECIMAL(18, 0),  
  PRIMARY KEY (Rno, Mno),  
  FOREIGN KEY (Mno) REFERENCES Medicine(Mno)  
}
```

■ RDBMS实现参照完整性时需要考虑的问题：

- ◆ 外码是否可以接受空值的问题
- ◆ 在被参照关系中删除元组时的问题
- ◆ 在参照关系中插入元组时的问题
- ◆ 修改关系中主码的问题



- 关系数据库系统都提供了定义和检查实体完整性、参照完整性和用户定义的完整性的功能
- 违反实体完整性规则和用户定义的完整性规则的操作
 - ◆ 一般是拒绝执行
- 违反参照完整性的操作
 - ◆ 拒绝执行
 - ◆ 或者接受这个操作，同时执行一些附加的操作，以保证数据库的状态正确



■ 受限策略 (RESTRICTED)

- ◆ 系统的默认方式。

- ◆ 当出现违背参照完整性规则的更新操作请求时，系统拒绝执行该操作。

■ ? 请分析：在RecipeDetail中新增、修改Mno分量，在Medicine中删除、修改Mno的分量值时，系统如何处理？

```
CREATE TABLE RecipeDetail {  
  Rno VARCHAR(10),  
  Mno VARCHAR(10) NOT NULL,  
  Mamount DECIMAL(18, 0),  
  PRIMARY KEY (Rno, Mno),  
  FOREIGN KEY (Mno) REFERENCES Medicine (Mno)  
}
```




■ 置空策略 (SET-NULL)

- ◆ 依照参照完整性规则，外码是可以取空值的。
- ◆ 但具体能否取空值，要根据应用环境的语义来定。

■ 示例

```
CREATE TABLE Doctor {  
  Dno VARCHAR(10) PRIMARY KEY,  
  Dname VARCHAR(50) NOT NULL,  
  Dsex VARCHAR(2),  
  Dage INT,  
  Ddeptno VARCHAR(10) REFERENCES Dept (DeptNo)  
  ON DELETE SET NULL ,  
  Dlevel VARCHAR(50),  
  Dsalary DECIMAL(18, 2)  
}
```



■ 级联策略（CASCADE）

- ◆ 不用拒绝用户操作请求的处理方式。
- ◆ 连带处理参照数据。

■ 示例：请分析主从表的连带操作？

```
CREATE TABLE RecipeDetail{  
  Rno VARCHAR(10),  
  Mno VARCHAR(10) NOT NULL,  
  Mamount DECIMAL(18, 0),  
  PRIMARY KEY(Rno, Mno),  
  FOREIGN KEY (Mno) REFERENCE Medicine(Mno)  
  ON DELETE CASCADE  
  ON UPDATE CASCADE  
}
```



■ 域约束

- ◆通过CREATE DOMAIN可以定义一个新的域；
- ◆通过对域进行约束可以达到对属性列取值的约束；
- ◆SQL92用一个特殊的关键字**VALUE**表示域的一个值。
- ◆统一约束，便于修改。

■ 定义域

```
CREATE DOMAIN SexVal CHAR(2)  
CHECK (VALUE IN('男', '女'));
```

■ 使用域

```
CREATE TABLE Patient{  
Pno VARCHAR(10),  
Pname VARCHAR(50) NOT NULL,  
Psex SexVal,  
Page INT,  
Pino VARCHAR(50),  
Pid VARCHAR(18),  
PRIMARY KEY(Pno)}
```



- 表级约束只关联一张表。虽然在使用CHECK语句时其表达式中可以引用其他关系表，但表级约束要求关联的表是非空的。
- 当约束条件涉及两张、甚至更多的关系表时，这种表级约束的缺陷就显现出来，不能满足需要。
- SQL中的断言可以解决多张关系表关联的约束定义。
- 前面介绍的域约束和参照完整性约束是断言的特殊形式，它们容易检测并适用于很多数据库应用。
- 当创建断言时，系统要检测其有效性。如果断言有效，则以后只有不破坏断言的数据库修改才被允许。如果断言比较复杂，则检测会带来相当大的开销。因此，使用断言应该特别小心。



■ 断言格式

CREATE ASSERTION <断言名> CHECK<谓词>

■ 断言示例

```
Create assertion salarycheck check(  
  Not exists(  
    Select * from Doctor x  
    Where Dsalary >= some ( select Dsalary from Doctor y  
    Where x.Deptno=y.Deptno and y.Dno =(  
      Select Manager from Dept  
      Where x.Deptno =Dept.Deptno)  
    )  
  )
```



- 任何时候都可以添加、修改、删除约束。
- 为了对约束进行修改、删除，有必要对约束命名。
- 约束命名：CONSTRAINT关键字后跟约束名称

■ 约束示例

```
CREATE TABLE RecipeDetail {  
  Rno VARCHAR(10),  
  Mno VARCHAR(10) CONSTRAINT notnullMno NOT NULL,  
  Mamount DECIMAL(18,0),  
  CONSTRAINT pkRecipeDetailRnoMno PRIMARY KEY (Rno, Mno),  
  CONSTRAINT fkRecipeDetailMnoMedicine FOREIGN KEY (Mno)  
    REFERENCES Medicine(Mno)  
  ON DELETE CASCADE  
  ON UPDATE CASCADE  
}
```



■ 删除约束

◆ **ALTER TABLE** RecipeDetail **DROP CONSTRAINT**
pkRecipeDetailRnoMno

■ 增加约束

◆ **ALTER TABLE** RecipeDetail **ADD CONSTRAINT**
pkRecipeDetailRnoMno PRIMARY KEY (Rno, Mno);

◆ **ALTER TABLE** RecipeDetail **ADD CONSTRAINT**
fkRecipeDetailMnoMedicine FOREIGN KEY (Mno) REFERENCES
Medicine (Mno);

◆ **ALTER TABLE** Doctor **ADD CONSTRAINT** checkPsex CHECK (Psex IN
('男', '女'));

◆ **ALTER DOMAIN** rfee DECIMAL (18, 2) **DROP CONSTRAINT** rfee_test;

■ 删除断言

◆ **DROP ASSERTION** salarycheck.



- 每当数据库中的数据发生变化时，DBMS将立即检查这些变化与约束是否被冲突，若有冲突DBMS会拒绝产生变化的数据库操作的执行。
- 任何约束都可以声明为DEFERRABLE（可延迟的）或NOT DEFERRABLE（不可延迟的）。
 - ◆ 一个不可延迟的约束在每条SQL语句执行后都必须校验是否违反约束规则。
 - ◆ 可延迟的约束意味着，当事务开始后，对约束的检查可以推迟到晚些时候，但不晚于当前事务的结束。
 - ◆ 如果约束是可推迟的，可进一步声明为INITIALLY IMMEDIATE（初始化立即执行）或INITIALLY DEFERRED（初始化延迟执行）。
 - 如果约束是INITIALLY IMMEDIATE，那么每条语句之后就检查它。
 - 如果约束是INITIALLY DEFERRED，那么只有在事务结尾才检查它。
 - 约束检查的时间可以用SET CONSTRAINTS命令修改。



数据库产品对完整性的支持



■ SQL Server、Oracle

- ◆ NOT NULL
- ◆ CHECK约束
- ◆ UNIQUE约束
- ◆ PRIMARY KEY约束
- ◆ FOREIGN KEY约束



本章小结



- 数据库的完整性是为了保证数据库中存储的数据是正确的，所谓正确的是指符合现实世界语义的要求
- DBMS完整性实现的机制
 - ◆ 完整性约束定义机制
 - ◆ 完整性检查机制
 - ◆ 违背完整性约束条件时DBMS应采取的动作
- 完整性机制的实施会极大地影响系统性能
- 不同数据库产品对完整性的支持策略和支持程度是不同的
 - ◆ 许多数据库管理系统对完整性机制的支持比对安全性的支持要晚得多、也弱得多
 - ◆ 数据库厂商对完整性的支持越来越好。不仅能保证实体完整性和参照完整性而且能在DBMS核心定义、检查和保证用户定义的完整性约束条件